

N100 FINANCIAL INTELLIGENCE PLATFORM

Analyst Guide

A practical guide to the data pipeline, analytics, dashboard, REST API, clustering, screening, testing and reporting workflow.

Project period: July–August 2026

Sprint 6 — API Server, Clustering & Final QA

1. Project Overview

The N100 Financial Intelligence Platform is an analytics system for analysing Nifty 100 companies using structured financial statements, computed financial ratios, valuation metrics, sector information, peer groups and supporting corporate documents.

The platform combines ETL and data-quality validation with statistical KPI computation, KMeans clustering, screening, portfolio statistics, a FastAPI REST layer and a browser-based dashboard. The objective is to make financial analysis repeatable, auditable and easy to consume.

Analysts can inspect individual companies, compare sectors and peer groups, apply financial filters, review historical performance and retrieve annual-report links. The system is designed around a SQLite analytical database and exposes the processed information through API endpoints.

2. System Architecture

The platform follows a layered architecture. Source Excel/CSV files form the raw data layer. ETL and normalisation scripts clean and standardise the data before loading it into SQLite. Analytical modules calculate ratios, growth metrics, quality indicators and clustering features.

The API layer uses FastAPI and exposes versioned endpoints under /api/v1. The dashboard consumes these endpoints using browser-side JavaScript. Reports and generated CSV/PDF files provide offline analytical outputs.

Typical flow: Source files → validation → normalisation → SQLite → KPI/analytics → FastAPI → dashboard/reporting.

3. Data Foundation and ETL

The source data contains company master information, profit and loss statements, balance sheets, cash flows, ratios, market capitalisation, sectors, peer groups, analysis fields, documents and pros/cons information.

ETL processing should preserve company identifiers and year relationships. Required columns are validated before loading, duplicate keys are controlled, numeric fields are normalised, and foreign-key relationships are checked.

Analysts should treat the SQLite database as the analytical source of truth after a successful ETL run. Raw source files should remain unchanged so that transformations can be reproduced.

4. Financial KPIs

Core KPIs include return on equity, return on capital employed, debt-to-equity, interest coverage, asset turnover, net profit margin, operating profit margin, revenue growth and free-cash-flow growth.

ROE and related ratios must be interpreted in context. Extremely high ROE values can result from very small or negative equity and should therefore be investigated rather than treated as automatically superior.

CAGR values represent compound growth over the configured historical period. Missing historical observations should not be interpreted as zero growth.

5. KMeans Clustering

The clustering workflow uses five financial features: `return_on_equity_pct`, `debt_to_equity`, `revenue_cagr_5yr`, `fcf_cagr_5yr` and `operating_profit_margin_pct`.

Missing feature values are imputed using sector medians before standardisation. `StandardScaler` places the features on a comparable scale. KMeans uses five clusters with `random_state=42` so the assignment is reproducible.

The elbow analysis evaluates candidate cluster counts and the resulting company labels are written to `output/cluster_labels.csv`. Each row contains `company_id`, `cluster_id`, `cluster_name` and `distance_from_centroid`.

Cluster names are analytical labels rather than investment recommendations. They should be interpreted together with the underlying feature means and medians.

6. Screener Usage

The screener is intended for filtering the Nifty 100 universe using financial quality and valuation criteria. Typical filters include minimum ROE, minimum ROCE, maximum debt-to-equity, sector, revenue CAGR, PAT CAGR, FCF and valuation multiples.

Start with broad filters and progressively narrow the universe. Always inspect the returned metrics rather than relying only on the number of companies returned.

A screener result is a research shortlist, not a buy/sell recommendation. Financial data can contain exceptional observations, accounting effects and sector-specific definitions.

7. Company Profile Analysis

Enter a company ticker such as TCS to retrieve its latest financial information. The company profile should be reviewed together with its historical financial series, valuation metrics, CAGR statistics and supporting documents.

Historical trends are generally more informative than a single-year number. Analysts should examine revenue, operating profit, net profit, margins, ROE, leverage and free cash flow across multiple years.

Use the annual-report links for detailed qualitative context, accounting notes, management commentary and explanations for unusual movements.

8. Sector Analysis

The sector endpoints provide company counts and aggregate financial statistics. Use sector analysis to avoid comparing companies with fundamentally different business models using identical thresholds.

For example, leverage and ROE should be interpreted differently for financial institutions than for technology or industrial companies. Sector medians are often more robust than simple averages when outliers are present.

The platform currently uses canonical sector names such as Information Technology. API consumers should use the exact names returned by the sectors endpoint.

9. Peer Analysis

Peer groups provide a more focused comparison set than the entire Nifty 100 universe. The platform includes groups such as IT Services, Automobiles, FMCG, Pharmaceuticals, Private Banks and others.

Percentile-based comparison helps show whether a company's KPI is strong or weak relative to its peer group. Analysts should inspect the peer definition before drawing conclusions.

Peer analysis is complementary to sector analysis: a sector can be broad, while a peer group can represent a more specific business model.

10. Portfolio Statistics

Portfolio statistics summarise KPI distributions across the company universe. Percentiles such as P10, P25, P50, P75 and P90 show the spread and help analysts choose sensible screening thresholds.

The median is especially useful when extreme observations are present. Standard deviation gives an indication of dispersion but should be interpreted with the KPI's distribution and sector composition in mind.

11. Market Capitalisation and Valuation

Historical valuation data includes market capitalisation, enterprise value, P/E, P/B, EV/EBITDA and dividend yield for available years.

Valuation multiples should be evaluated against earnings quality, growth, leverage, sector norms and historical ranges. A low multiple is not automatically cheap, and a high multiple is not automatically expensive.

Missing historical valuation values are expected where source data is unavailable and should remain missing rather than being converted to zero.

12. REST API

The FastAPI server is exposed under the `/api/v1` prefix. The health endpoint provides a quick operational check. Company, financial, screener, sector, peer, market-cap, portfolio and document endpoints provide structured JSON responses.

Example requests include `GET /api/v1/health`, `GET /api/v1/companies`, `GET /api/v1/financials/TCS/latest`, `GET /api/v1/financials/TCS/cagr`, `GET /api/v1/financials/TCS/valuation` and `GET /api/v1/companies/TCS/documents`.

The generated `docs/openapi.json` file provides the machine-readable API specification. The interactive API documentation can be inspected through the FastAPI `/docs` route while the server is running.

13. Dashboard Operation

The dashboard is served from the src/dashboard directory using a local HTTP server. The browser application communicates with FastAPI at `http://127.0.0.1:8000/api/v1`.

Start the API first, then start the dashboard server. Confirm that the dashboard reports API Connected before using company and screener functions.

If the dashboard shows API Offline, verify that FastAPI is running on port 8000 and that the browser is loading the correct app.js configuration.

14. Testing and Quality Assurance

The project includes ETL, KPI, data-quality and API tests. The final QA target is at least 60 collected tests with zero failures.

Important acceptance checks include the 92-company count, foreign-key validation, ratio row count, CAGR spot checks, API health, TCS historical coverage, cluster assignment coverage and document completeness.

HTML pytest output should be retained as a reproducible QA artifact. Failures should be fixed at the source rather than hidden by changing test expectations.

15. Troubleshooting

API will not start: activate the virtual environment, verify dependencies, confirm port 8000 is free and start the FastAPI application with uvicorn.

Dashboard cannot connect: confirm FastAPI is running, check the API_BASE value in src/dashboard/app.js and reload the browser.

Sector not found: use the exact canonical sector returned by /api/v1/sectors. For example, use Information Technology instead of IT when that is the stored sector name.

Missing report files: check the expected output/report path and rerun the corresponding generation script. Keep output and outputs directory naming consistent.

16. Recommended Analyst Workflow

First check API health and data availability. Second select the relevant sector or peer group. Third run the screener with conservative filters. Fourth inspect individual company history and valuation. Fifth review peer and sector context. Finally open supporting annual reports for qualitative validation.

Document the filters and date/version of the dataset used for any analysis intended for presentation or decision support.

17. Runbook

Activate the environment with `source venv/bin/activate`. Start the FastAPI service on port 8000. Start the dashboard server on port 5500 when using the static dashboard. Verify `/api/v1/health`, then open the dashboard.

For development, regenerate analytical outputs after changing the source data or KPI logic. Run the relevant tests before using the regenerated reports.

18. Final Deliverables and Sign-Off

Final project sign-off should verify the required CSV outputs, plots, API implementation, OpenAPI specification, pytest report, analyst guide, tearsheets, validation report, pros/cons output and final archive.

Acceptance gates should be recorded as PASS or FAIL with evidence paths or command outputs. Any missing deliverable should be resolved before declaring Sprint 6 complete.